# Классы объектов. Компоненты и их свойства.

# Определение класса

```
class имя_класса
{
    // компоненты класса
};
```

# Определение класса

```
class Person { };

int main()
{

}
```

# Создание объекта

```
class Person
{

};
int main()
{
    Person person;
}
```

# Добавление метода

```
class Person
{
public:
    std::string name;
    unsigned age;
    void print()
    {
        std::cout << "Name: " << name << "\tAge: " << age << std::endl;
    }
};
int main()
{
    Person person;
    // устанавливаем значения полей класса
    person.name = "Tom";
    person.age = 38;
    // вызываем функцию класса
    person.print();
}
```

# Объект

Через точку мы можем обратиться к его переменным и функциям.

```
объект.компонент
```

Например, мы можем установить значения полей класса

```
person.name = «Tom";
person.age = 38;
```

Ну и также мы можем вызывать функции у объекта:

```
person.print();
```

# Получение значений свойств объектов

```
Person person;
    // устанавливаем значения полей класса
    person.name = "Bob";
    person.age = 42;
    // получаем значения полей
    std::string username = person.name;
    unsigned userage = person.age;
    // выводим полученные данные на консоль
    std::cout << "Name: " << username << "\tAge: " << userage <<
std::endl;
```

# Значения по умолчанию

```
class Person
{
public:
    std::string name = "Noname";
    unsigned age = 18;
    void print()
    {
        std::cout << "Name: " << name << "\tAge: " << age << std::endl;
    }
};
int main()
{
    Person person;
    person.print(); // Name: Noname Age: 18
}
```

# Конструкторы и инициализация объектов

# Что такое конструктор

Конструкторы представляют специальную функцию, которая имеет то же имя, что и класс, которая не возвращает никакого значения и которая позволяют инициалилизировать объект класса во время го создания и таким образом гарантировать, что поля класса будут иметь определенные значения. При каждом создании нового объекта класса вызывается конструктор класса.

# Пример

```
class Person
{
public:
    string name;
    unsigned age;
    void print()
    {
        cout << "Name: " << name << "\tAge: " << age << endl;
    }
    Person(string p_name, unsigned p_age)
    {
        name = p_name;
        age = p_age;
        cout << "Person has been created" << endl;
    }
};
int main()
{
    Person tom("Tom", 38);  // создаем объект - вызываем конструктор
    tom.print();
}
```

# Пример

```
Person(string p_name, unsigned p_age)
{
    name = p_name;
    age = p_age;
    cout << "Person has been created" << endl;
}
```

# Пример 2

## Определение нескольких конструкторов

```
class Person
{
    std::string name{};
    unsigned age{};
public:
    void print()
    {
        std::cout << "Name: " << name << "\tAge: " << age << std::endl;
    }
    Person(std::string p_name, unsigned p_age)
    {
        name = p_name;
        age = p_age;
    }
    Person(std::string p_name)
    {
        name = p_name;
        age = 18;
    }
    Person()
    {
        name = "Undefined";
        age = 18;
    }
};
```

# Параметр по умолчанию

```
Person(std::string p_name = "Undefined", unsigned p_age = 18)
    {
        name = p_name;
        age = p_age;
    }
```

# Управление доступом. Инкапсуляция

# Что это

Класс может определять различное состояние, различные функции. Однако не всегда желательно, чтобы к некоторым компонента класса был прямой доступ извне. Для разграничения доступа к различным компонентам класса применяются **спецификаторы доступа**

# Пример

```
class Person
{
private:
    std::string name;
    unsigned age;
public:
    void print()
    {
        std::cout << "Name: " << name << "\tAge: " << age << std::endl;
    }
    Person(std::string p_name, unsigned p_age)
    {
        name = p_name;
        age = p_age;
    }
};
```